Introduction to BASIC (bcc)

Nov 1, 2026 · @Scott

Welcome

bcc turns plain BASIC source files into native programs for HP-UX and Linux, and this guide teaches you to write them. It assumes you have used a computer but have never programmed.

The language is a structured BASIC modelled on QuickBASIC and FreeBASIC. You do not need line numbers. Programs are organised with `SUB` and `FUNCTION` blocks, and errors are handled with `TRY / CATCH`.

Behind the scenes, bcc translates your `.bas` file into C and hands it to `gcc`, which builds the final binary. You never have to look at the C, but it explains why programs run fast.

How to use this guide:

- Read the sections in order. Each one builds on the last.
- Type every example yourself and run it. Reading code teaches far less than running it.
- Keep the reference manual (`REFERENCE.md`) open for the full details. This guide covers what a newcomer needs first.

All you need is bcc itself. It ships with `bcc-ide`, a full-screen terminal IDE, so you do not need a separate editor. Run `bcc-ide` to start it, or `bcc-ide hello.bas` to open a file. If you prefer your own editor, `nano` and `vi` work too. If `bcc` runs and prints a usage message, you are ready.

Your first program

A BASIC program is a text file ending in `.bas`, and compiling it takes one command. Create `hello.bas` in `bcc-ide` (run `bcc-ide hello.bas`) or in any text editor:

```
REM My first program
PRINT "Hello, world!"
```

Compile and run it:

```
bcc hello.bas
./hello
```

The first command produces an executable named `hello` in the current directory. The second runs it, and you should see `Hello, world!`.

A slightly more useful version asks your name:

```
DIM name AS STRING
INPUT "What is your name? ", name
PRINT "Nice to meet you, "; name
```

The bcc options you will use most

Option	What it does
`-o outfile`	Names the output program (default: the `.bas` file's base name)
`-c`	Writes the generated `.c` file and stops, without building
`-v`	Prints each pipeline step and keeps the temporary `.c` file
`--lex`	Dumps the token stream, useful for debugging syntax errors
`--parse`	Dumps the parse tree
`-O0` / `-O1` / `-O2`	Sets the gcc optimisation level (default `-O2`)

bcc exits with 0 on success, 1 on a compile or link error, and 2 for a usage error.

When the compiler reports errors, fix the first one and recompile. One mistake often triggers a chain of later messages.

How a program is laid out

BASIC is line-oriented: one line holds one statement, and the newline ends it. Everything else in this section follows from that rule.

Several statements on one line. Separate them with a colon:

```
a = 1 : b = 2 : PRINT a + b
```

Long lines. End a line with an underscore to continue it on the next:

```
total = (firstValue + _
         secondValue) * 3
```

Comments. Use `REM` or an apostrophe. Comments are ignored by the compiler and exist for people:

```
REM This whole line is a comment
x = 5   ' so is everything after the apostrophe
```

Case does not matter. `PRINT`, `Print` and `print` are the same, and so are `Total` and `TOTAL`.

Labels. A name followed by a colon at the start of a line marks a place you can jump to with `GOTO` or `GOSUB`:

```
Again:
    PRINT i
    i = i + 1
    IF i < 10 THEN GOTO Again
```

The shape of a whole program

A source file can contain, in any sensible order:

- an optional `OPTION BASE` line (it must come before any `DIM`),
- `TYPE` definitions,
- `SUB` and `FUNCTION` definitions,
- the main program: ordinary statements at the top level.

Execution starts at the first top-level statement. The bodies of `SUB` and `FUNCTION` blocks are skipped over until something calls them. Put `END` anywhere to stop the program immediately.

Variables and data types

A variable is a named box that holds a value, and every box has a type that says what kind of value fits. bcc has five types.

Type	Suffix	Holds	Range or precision
`INTEGER`	`%`	whole numbers	-2,147,483,648 to 2,147,483,647
`LONG`	`&`	whole numbers	same as `INTEGER` on this platform
`SINGLE`	`!`	decimals	about 7 digits
`DOUBLE`	`#`	decimals	about 15 digits
`STRING`	`$`	text	any length

On 32-bit PA-RISC, `INTEGER` and `LONG` are both 32 bits, so `LONG` gives no extra range. Use `DOUBLE` when you need precision.

Declaring variables with DIM

Declare a variable with `DIM name AS type`:

```
DIM count AS INTEGER
DIM price AS DOUBLE
DIM city AS STRING
DIM a AS INTEGER, b AS INTEGER, c AS DOUBLE
```

Then assign with `=`:

```
count = 10
price = 19.95
city = "Denver"
```

Strings go in double quotes. To put a quote mark inside a string, double it: `"say ""hi"""`.

Declaring by suffix, and the SINGLE trap

You can skip `DIM` and let a suffix choose the type:

```
n% = 42
name$ = "Alice"
ratio# = 3.14159265358979
```

A variable with no `DIM` and no suffix becomes `SINGLE` and triggers a compiler warning. A misspelled name silently creates a new variable this way, so declare everything with `DIM`.

Scope

A variable declared inside a `SUB` or `FUNCTION` is local and disappears when that block ends. To stay safe, treat a top-level variable as invisible inside a `SUB` or `FUNCTION` unless you pass it in as a parameter or declare it with `DIM SHARED`:

```
DIM SHARED globalCounter AS INTEGER
```

`OPTION BASE 0` or `OPTION BASE 1` sets the first index of arrays that you declare without explicit bounds. It must come before any `DIM`.

Operators and expressions

An expression combines values and operators into a new value, and BASIC follows the same arithmetic rules you learned in school. Here are the operators:

Operator	Meaning	Example	Result
`+` `-` `*`	add, subtract, multiply	`6 * 7`	42
`/`	division with decimals	`7 / 2`	3.5
`\`	whole-number division (truncates)	`7 \ 2`	3
`MOD`	remainder	`7 MOD 3`	1
`^`	power	`2 ^ 10`	1024
`&`	join strings	`"Hello" & " " & "World"`	Hello World

Two details trip up beginners. Use `\`, not `/`, when you want a whole-number answer. And join text with `&`; the `+` sign is for numbers.

Comparing values

The comparison operators are `=`, `<>` (not equal), `<`, `>`, `<=` and `>=`. A comparison gives `-1` for true and `0` for false. Strings compare in ASCII order, so `"apple" < "banana"` is true.

Combining conditions

`AND`, `OR`, `NOT` and `XOR` combine true/false values:

```
IF age >= 18 AND hasTicket THEN PRINT "Welcome"
```

On whole numbers the same operators work bit by bit, which is useful for flags and masks.

Order of evaluation

Operators run from the top of this list to the bottom:

1. `^`
2. unary minus and plus (`-x`)
3. `*` `/` `\` `MOD`
4. `+` `-`
5. `&`
6. comparisons
7. `NOT`
8. `AND`
9. `OR` `XOR`

When in doubt, add parentheses: `(a + b) * c` is always clear.

Mixing types

Whole number with whole number gives a whole number, except for `/`, which gives a decimal. If either side is a decimal, the whole expression becomes decimal. Comparing two different number types promotes both to `DOUBLE`.

Making decisions

`IF` lets your program choose what to do based on a condition. The short form fits on one line:

```
IF score >= 50 THEN PRINT "Pass" ELSE PRINT "Fail"
```

For more than one statement, use the block form, which ends with `END IF`:

```
IF score >= 90 THEN
    PRINT "Excellent"
ELSEIF score >= 50 THEN
    PRINT "Pass"
ELSE
    PRINT "Fail"
END IF
```

You can chain as many `ELSEIF` branches as you like. The first true condition wins, and the rest are skipped.

SELECT CASE

When you compare one value against many possibilities, `SELECT CASE` reads better than a long `IF` chain:

```
DIM day AS INTEGER
INPUT "Day number (1-7): ", day

SELECT CASE day
    CASE 1
        PRINT "Monday"
    CASE 6, 7
        PRINT "Weekend"
    CASE 2 TO 5
        PRINT "Midweek"
    CASE IS > 7
        PRINT "That is not a day"
    CASE ELSE
        PRINT "Out of range"
END SELECT
```

A `CASE` can hold a single value, a comma list, a range with `TO`, or a comparison with `IS`. Cases are tested from the top, and the first match runs. `SELECT CASE` also works on strings.

GOTO and GOSUB

`GOTO Label` jumps to a label. `GOSUB Label` jumps there and comes back at the next `RETURN`. Both work, but `SUB` and `FUNCTION` (covered below) make programs much easier to follow, so prefer them.

Loops

A loop repeats a block of statements, and BASIC offers three kinds. Pick the one that matches what you know in advance.

FOR / NEXT: when you know how many times

```
FOR i = 1 TO 5
    PRINT i; " squared is "; i * i
NEXT i
```

Add `STEP` to count by a different amount, or backwards:

```
FOR i = 10 TO 0 STEP -2
    PRINT i
NEXT i
```

The step defaults to 1. The variable name after `NEXT` is optional.

WHILE / WEND: repeat while something is true

```
DIM n AS INTEGER
n = 1
WHILE n < 100
    n = n * 2
WEND
PRINT n
```

The test happens first, so the body may run zero times.

DO / LOOP: the flexible one

`DO` loops can test at the top or the bottom, with `WHILE` or `UNTIL`:

```
DO WHILE n > 0      ' test first, loop while true
    n = n - 1
LOOP

DO                  ' test last, so the body runs at least once
    INPUT "Enter a positive number: ", n
LOOP UNTIL n > 0    ' stop when the condition becomes true
```

A bare `DO ... LOOP` with no test runs forever, until you leave it with `EXIT DO`.

Leaving a loop early

`EXIT FOR` and `EXIT DO` jump out of the nearest enclosing loop:

```
FOR i = 1 TO 100
    IF i * i > 50 THEN EXIT FOR
NEXT i
PRINT "First square over 50 comes from "; i
```

Loops can be nested, and an `EXIT` only leaves the innermost one.

Subroutines and functions

A `SUB` or `FUNCTION` is a named block of code you write once and use many times. The difference is that a `FUNCTION` hands back a value and a `SUB` does not.

SUB

```
SUB Greet(who AS STRING)
    PRINT "Hello, "; who; "!"
END SUB

Greet "Ada"
CALL Greet("Grace")
```

You can call a `SUB` by writing its name followed by the arguments, or with `CALL` and parentheses. `EXIT SUB` returns early.

FUNCTION

Set a function's result by assigning to its own name:

```
FUNCTION Square(n AS DOUBLE) AS DOUBLE
    Square = n * n
END FUNCTION

PRINT Square(12)
```

Call a function inside an expression, as in `x = Square(a) + 1`. `EXIT FUNCTION` returns early with whatever value the name currently holds.

Passing values in

Mode	What happens	Default for
`BYVAL`	the block gets a copy; the caller's variable never changes	numbers
`BYREF`	the block works on the caller's own variable	strings and arrays

```
SUB AddOne(BYREF n AS INTEGER)
    n = n + 1
END SUB

DIM x AS INTEGER
x = 5
AddOne x
PRINT x       ' prints 6
```

Arrays are always passed by reference. Write the parameter with empty parentheses, and pass the array the same way:

```
SUB FillArray(arr() AS INTEGER, n AS INTEGER)
    DIM i AS INTEGER
    FOR i = 0 TO n - 1
        arr(i) = i * 2
    NEXT i
END SUB

DIM data(100) AS INTEGER
FillArray data(), 100
```

Recursion

A block can call itself. Every recursive function needs a stopping case:

```
FUNCTION Factorial(n AS INTEGER) AS LONG
    IF n <= 1 THEN
        Factorial = 1
    ELSE
        Factorial = n * Factorial(n - 1)
    END IF
END FUNCTION
```

Because `LONG` is 32 bits here, factorials overflow past 12!.

Arrays

An array is a numbered row of boxes that share one name and one type. Reach each box with an index in parentheses.

```
DIM scores(4) AS INTEGER
scores(0) = 90
scores(1) = 75
PRINT scores(0) + scores(1)
```

The number in `DIM` is the highest index, not the count. `DIM scores(4)` makes five boxes, numbered 0 to 4. To pick your own range, write it out:

```
DIM month(1 TO 12) AS STRING
DIM grid(1 TO 5, 1 TO 5) AS DOUBLE    ' two dimensions
```

Using an index outside the range raises runtime error 9, "Subscript out of range" (see the section on errors).

Fixed-size and sized-at-runtime arrays

When the bounds are plain numbers, the array has a fixed size known at compile time. When a bound is a variable, bcc allocates the array on the heap while the program runs:

```
DIM n AS INTEGER
INPUT "How many values? ", n
DIM values(n) AS INTEGER      ' size decided by the user
```

This is how the prime sieve later in this guide sizes itself to the number you type.

Resizing with REDIM

`REDIM` changes the size of an array that was declared with variable bounds:

```
REDIM values(newSize) AS INTEGER            ' resize; old contents are lost
REDIM PRESERVE values(newSize) AS INTEGER   ' resize; keep what fits
```

With `PRESERVE`, only the last dimension may change.

Looping over an array

```
DIM total AS INTEGER, i AS INTEGER
FOR i = 0 TO 4
    total = total + scores(i)
NEXT i
PRINT "Total: "; total
```

Variable-bound arrays are not guaranteed to start out zeroed, so set every element yourself before you read it, as the sieve does.

To hand an array to a `SUB`, write `FillArray data(), 100`. See the section on subroutines for the matching parameter.

User-defined types

A `TYPE` bundles several related values under one name, so a record such as a point or a contact travels as a single unit. Define it once at the top level:

```
TYPE Point
    x AS DOUBLE
    y AS DOUBLE
END TYPE
```

Then declare variables of that type and reach each field with a dot:

```
DIM p AS Point
p.x = 3.14
p.y = 2.72
PRINT p.x; p.y
```

Types inside types

A field can itself be a user-defined type:

```
TYPE Rectangle
    topLeft AS Point
    width   AS DOUBLE
    height  AS DOUBLE
END TYPE

DIM r AS Rectangle
r.topLeft.x = 0
r.topLeft.y = 0
r.width = 100
r.height = 50
```

Define `Point` before `Rectangle`, since `Rectangle` uses it.

Arrays of records

```
DIM points(10) AS Point
points(1).x = 1.0
points(1).y = 2.0
```

You can pass a typed variable to a `SUB` or `FUNCTION`, by value or by reference. Typed records also work with random-access files, covered in the file section.

Talking to the user

Programs become useful when they show results and read answers, and BASIC does both with a few short statements.

PRINT

```
PRINT "Total:"; total       ' semicolon: no gap between items
PRINT a, b                 ' comma: jump to the next tab stop
PRINT "Loading...";        ' trailing semicolon: no new line
PRINT                      ' blank line
```

Positive numbers print with a leading space, and negative numbers print with a minus sign. To avoid the space when you build text, use `TRIM$(STR$(n))`.

PRINT USING: formatted numbers

`PRINT USING` lays out numbers and text by a format string:

```
PRINT USING "###.##"; 3.14159          ' prints   3.14
PRINT USING "$$#,###.##"; 1234.5        ' prints $1,234.50
PRINT USING "\     \ ###"; name; qty    ' fixed-width text, then a number
```

Symbol	Meaning
`#`	one digit position
`.`	decimal point
`,`	thousands separator inside a `#` mask
`+` `-`	sign shown leading or trailing
`$` `$$`	fixed or floating dollar sign
`**`	fill unused leading positions with `*`
`!` `\ \` `&`	text fields: first character, fixed width, whole string

A trailing semicolon after the last item suppresses the new line. A few GW-BASIC features are not supported, such as the `_` escape and the `%` overflow marker. The reference manual lists them.

INPUT

```
INPUT "Your name: ", name        ' comma: shows only your prompt
INPUT "Your age"; age            ' semicolon: adds "? " after the prompt
INPUT x, y                      ' several comma-separated values
```

With a comma after the prompt you control the whole line, so end the prompt with a space or a colon. Strings do not need quote marks unless they contain commas.

LINE INPUT

`LINE INPUT` reads a whole line as one string, commas and all:

```
LINE INPUT "Address: ", addr
```

Controlling the screen

Statement	What it does
`CLS`	clears the screen
`LOCATE row, col`	moves the cursor (1-based)
`COLOR fg, bg`	sets text colours; 0-15 foreground, 0-7 background
`BGCOLOR bg`	sets the background only

```
COLOR 14, 1
PRINT "Warning!"
COLOR 7
```

`TERMROWS` and `TERMCOLS` give the terminal size, so you can lay out text to fit.

Reading single keys

`INKEY$` checks the keyboard without waiting. It returns an empty string when no key is pressed:

```
DIM k AS STRING
DO
    k = INKEY$
    IF k = CHR$(27) THEN EXIT DO     ' ESC quits
    IF LEN(k) > 0 THEN PRINT "You pressed: "; k
LOOP
```

Arrow and function keys arrive as `CHR$(0)` followed by a scan code, such as 72 for the up arrow.

Command-line arguments and the shell

```
first$ = COMMAND$(1)            ' first argument; COMMAND$(0) is the program
name
all$ = COMMAND$                 ' every argument joined by spaces

SHELL "cp data.txt backup.txt" ' run a command and wait
IF ERRLEVEL <> 0 THEN PRINT "copy failed"
host$ = SHELL$("uname -n")      ' capture a command's output as a string
```

`ERRLEVEL` holds the exit code of the last `SHELL`. To send text to the printer, use `LPRINT`, which works like `PRINT` and submits the job through the `lp` spooler when you run `LPRINT CLOSE` or the program ends.

Working with files

A program opens a file under a number from 1 to 255, uses that number for every read or write, and closes it when finished. The pattern is always the same:

```
OPEN "notes.txt" FOR OUTPUT AS #1
PRINT #1, "First line"
PRINT #1, "Second line"
CLOSE #1
```

File modes

Mode	Use it to
`INPUT`	read a text file from the start
`OUTPUT`	write a text file, creating or emptying it
`APPEND`	add to the end of an existing text file
`RANDOM`	read and write fixed-length records
`BINARY`	read and write raw bytes

`CLOSE #1` closes one file, and a bare `CLOSE` closes them all. Be careful with `OUTPUT`: it erases whatever was in the file.

Saving and loading values

`WRITE #` stores values in comma-separated form, with strings in quotes. `INPUT #` reads them back in the same order:

```
OPEN "people.csv" FOR OUTPUT AS #1
WRITE #1, "Alice", 42, 3.14
CLOSE #1

DIM name AS STRING, age AS INTEGER, score AS DOUBLE
OPEN "people.csv" FOR INPUT AS #1
INPUT #1, name, age, score
CLOSE #1
PRINT name; age; score
```

Reading until the end

`EOF(n)` tells you when a file has run out. It returns -1 (true) at the end:

```
DIM textLine AS STRING
OPEN "notes.txt" FOR INPUT AS #1
DO WHILE EOF(1) = 0
    INPUT #1, textLine
    PRINT textLine
LOOP
CLOSE #1
```

`INPUT #` splits on commas, so a line containing commas arrives in pieces. Keep text lines comma-free, or read files that `WRITE #` created.

`LOF(n)` gives the file size in bytes and `LOC(n)` the current position.

Random-access records

For data you want to jump around in, such as a customer list, use `RANDOM` files. Every record has the same length, so record number *n* sits at a known place. `GET` reads a record and `PUT` writes one, counting from record 1:

A random file works with a `TYPE` record variable (`GET #1, 1, rec` and `PUT #1, 1, rec`) or with a `FIELD` buffer, shown next.

A `FIELD` buffer, which maps parts of each record to string variables:

```
OPEN "emp.dat" FOR RANDOM AS #1 LEN = 30
FIELD #1, 20 AS empName$, 2 AS empAge$, 8 AS empSalary$

LSET empName$   = "Alice"       ' left-justify, pad with spaces
LSET empAge$    = MKI$(34)      ' pack a number into binary text
LSET empSalary$ = MKD$(72500.50)
PUT #1, 1                       ' write record 1

GET #1, 1                       ' read record 1 back
PRINT RTRIM$(empName$); " is "; CVI(empAge$)
CLOSE #1
```

`LSET` and `RSET` left- or right-justify text into a field. The `MK` functions (`MKI$`, `MKL$`, `MKS$`, `MKD$`) pack numbers into strings, and the `CV` functions (`CVI`, `CVL`, `CVS`, `CVD`) unpack them. Make the field widths add up to the record length given in `LEN =`; the default length is 128.

Handling errors

Things go wrong at runtime: a file is missing, a number is divided by zero, an index is out of range. Without protection the program stops. With `TRY / CATCH` you decide what happens instead.

TRY / CATCH

```
TRY
    OPEN "settings.dat" FOR INPUT AS #1
    PRINT "File opened"
    CLOSE #1
CATCH e AS STRING
    PRINT "Could not open the file: "; e
END TRY
```

If any statement in the `TRY` block fails, execution jumps straight to `CATCH`, and the variable after `CATCH` receives the error message. The variable is optional. `TRY` blocks can be nested, and the innermost one catches first.

Here is one that catches an out-of-range index:

```
TRY
    DIM a(5) AS INTEGER
    PRINT a(100)
CATCH e AS STRING
    PRINT "Error: "; e
END TRY
```

ON ERROR GOTO: the older way

If you meet older programs, you will see `ON ERROR GOTO`. It sends every unhandled error to a label, where `ERR` is the error number and `ERROR$(n)` its message:

```
ON ERROR GOTO Oops
x = 1 \ 0
PRINT "carried on"
GOTO Done

Oops:
    PRINT "Error "; ERR; ": "; ERROR$(ERR)
    RESUME NEXT

Done:
    PRINT "finished"
```

`RESUME NEXT` continues after the statement that failed. A plain `RESUME`, which retries that statement, is not supported. `ON ERROR GOTO 0` turns the handler off. For new programs, prefer `TRY / CATCH`.

Common error codes

Code	Message	Typical cause
5	Illegal function call	`SQR` of a negative, `LOG` of zero or less
7	Out of memory	an array that is too big
9	Subscript out of range	index outside the array bounds
11	Division by zero	`/` or `\` with a zero divisor
52	Bad file number	using a file that is not open

Code	Message	Typical cause
53	File not found	opening a file that does not exist
62	Input past end of file	reading beyond the last record

The reference manual lists every code.

Built-in function quick reference

bcc includes functions for text, maths and conversion, and you call them inside expressions. String positions count from 1, not 0.

Text

Function	Returns
`LEN(s$)`	number of characters
`LEFT$(s$, n)` / `RIGHT$(s$, n)`	the first or last `n` characters
`MID$(s$, start, len)`	`len` characters from position `start` (leave out `len` for the rest)
`UCASE$(s$)` / `LCASE$(s$)`	upper- or lower-case copy
`TRIM$(s$)` / `LTRIM$(s$)` / `RTRIM$(s$)`	copy without spaces at both ends, the start, or the end
`INSTR(hay$, needle$)`	position where `needle$` first appears, or 0
`STR$(n)`	a number as text (positives get a leading space)
`VAL(s$)`	text as a number, or 0 if it is not one
`CHR$(n)` / `ASC(s$)`	character for a code, and code of the first character
`SPACE$(n)` / `STRING$(n, ch)`	`n` spaces, or `n` copies of a character
`HEX$(n)` / `OCT$(n)`	base-16 or base-8 text

`MID$` can also sit on the left of an assignment to overwrite part of a string in place:

```
s = "Hello World"
MID$(s, 7, 5) = "BASIC"      ' s is now "Hello BASIC"
```

Maths

Function	Returns
`SQR(x)`	square root (error 5 if `x` is negative)
`ABS(x)`	absolute value
`INT(x)`	rounds down: `INT(-3.2)` is -4
`FIX(x)`	cuts off the decimals: `FIX(-3.7)` is -3
`SGN(x)`	-1, 0 or 1
`SIN` `COS` `TAN` `ATN`	trigonometry, in radians
`LOG(x)` / `EXP(x)`	natural log, and e to the power `x`
`RND`	a random number from 0 up to, but not including, 1

Call `RANDOMIZE` once at the start of a program to seed the random numbers from the clock. Without it, a program may repeat the same sequence each run. A random whole number from 1 to 6 looks like this:

```
RANDOMIZE
die = INT(RND * 6) + 1
```

Converting between types

Function	Returns
`CINT(x)` / `CLNG(x)`	rounded to the nearest whole number
`CSNG(x)` / `CDBL(x)`	as a single- or double-precision decimal

`CINT` uses banker's rounding, so exact halves go to the nearest even number.

Worked example: a prime number sieve

This program asks for a limit and prints every prime number up to it, and it uses nearly every idea from the sections above. It uses the Sieve of Eratosthenes, an old method that crosses out the multiples of each prime and leaves the primes standing. The finished program is `primes3.bas`; here is how it works.

Step 1: ask how far to go

```
DIM limit AS LONG
INPUT "Find primes up to what number? ", limit
```

The comma after the prompt means no `?` is added, so the prompt ends the way you wrote it. Before it runs the sieve, the program rejects limits below 2 and limits above a safety cap of 50,000,000.

Step 2: build the sieve

```
DIM sieve(n) AS INTEGER          ' one flag per number: 0 = prime, 1 =
composite

FOR i = 0 TO n
    sieve(i) = 0                 ' start by assuming everything is prime
NEXT i

root = INT(SQR(n))
FOR i = 2 TO root
    IF sieve(i) = 0 THEN         ' i is prime, so cross out its multiples
        FOR j = i * i TO n STEP i
            sieve(j) = 1
        NEXT j
    END IF
NEXT i
```

Three details are worth noticing:

- The array bound `n` is a variable, so the array is sized at runtime, as the arrays section described. The loop sets every element to 0 itself, because it cannot assume the memory starts clean.
- The outer loop stops at the square root of `n`. Any composite number up to `n` has a factor no bigger than that, so there is nothing left to cross out beyond it.

- The inner loop starts at `i * i`, because smaller multiples were already crossed out by smaller primes. `STEP i` then hops from one multiple to the next.

Step 3: count and print

```
perLine = TERMCOLS \ 13           ' each cell: 11 characters plus a 2-space
gap

FOR i = 2 TO n
    IF sieve(i) = 0 THEN
        count = count + 1
        lastPrime = i
        IF showList <> 0 THEN
            PRINT USING "###,###,###"; TRIM$(STR$(i));
            PRINT "  ";
            col = col + 1
            IF col >= perLine THEN
                PRINT
                col = 0
            END IF
        END IF
    END IF
NEXT i
```

Every number still marked 0 is prime. `PRINT USING` right-justifies each prime in an 11-character field with thousands separators, so the columns line up, and the trailing semicolon keeps the cursor on the same line. The two spaces after it separate the columns. When `col` reaches `perLine`, the terminal width divided by 13, the program starts a new line.

Step 4: make the numbers readable

`PRINT USING` suits a fixed grid, but in a sentence it would pad short numbers with blanks. The program therefore has a small function, `Commas`, that inserts the separators itself:

```
FUNCTION Commas(n AS LONG) AS STRING
    DIM s AS STRING
    DIM result AS STRING
    DIM i AS INTEGER
    DIM k AS INTEGER
    s = TRIM$(STR$(n))
    result = ""
    k = 0
    FOR i = LEN(s) TO 1 STEP -1
        IF k > 0 AND k MOD 3 = 0 THEN result = "," & result
        result = MID$(s, i, 1) & result
        k = k + 1
    NEXT i
    Commas = result
END FUNCTION
```

The loop walks the digits from right to left, adding a comma before every third one. This one function shows string handling (`MID$`, `&`), `STEP -1` and a `FUNCTION` that returns text. The summary lines then read cleanly:

```
PRINT "Range searched : 2 to "; Commas(n)
PRINT "Primes found   : "; Commas(count)
PRINT "Largest prime  : "; Commas(lastPrime)
```

Step 5: keep the calculation safe

The sieve lives in a `SUB` named `RunSieve`, and its work sits inside `TRY / CATCH`. If the machine runs out of memory for a very large limit, the user sees a message instead of a crash.

The main program wraps everything in a `DO ... LOOP WHILE` so the user can run again, and it uses `INPUT` for the Y/N answers.

Try it

```
bcc primes3.bas
./primes3
```

Enter `30` and answer `Y` to list. The program prints the heading `Primes up to 30`, then the ten primes (2, 3, 5, 7, 11, 13, 17, 19, 23 and 29) in columns, then a summary: range 2 to 30, 10 primes found, largest prime 29. Try `1000000` with the list turned off to see the

separators at work: 78,498 primes, the largest being 999,983.

Once it works, try these changes:

1. Print only the primes between two numbers the user chooses.
2. Add a function `IsPrime(n)` that tests one number by trial division, and compare its answer with the sieve.
3. Write the primes to a file with `PRINT #`.

Gotchas and platform notes

Most beginner problems come from a handful of habits, and knowing them in advance saves hours.

- **`LONG` is not bigger than `INTEGER` here.** On 32-bit PA-RISC both hold about plus or minus 2.1 billion. For larger or fractional numbers, use `DOUBLE`.
- **Undeclared variables become `SINGLE`.** A typo in a name creates a new variable and only a warning. Declare everything with `DIM`.
- **Arrays are zero-based unless you say otherwise.** `DIM a(10)` has 11 elements, 0 to 10. Use `OPTION BASE 1` or `1 TO n` bounds for one-based arrays.
- **`/` and `\` differ.** `/` gives decimals and `\` gives whole numbers.
- **`+` adds numbers and `&` joins text.**
- **A plain `RESUME` does not exist.** Use `RESUME NEXT`, or better, `TRY / CATCH`.
- **Fix the first compiler error first.** Later messages are often side effects of the first.
- **Case does not matter, but spelling does.** `Total` and `TOTAL` are one variable, and `Totl` is a different one.

Features not in this dialect

If you have used QuickBASIC before, these will not work:

- line numbers (use labels instead),
- `DATA` and `READ`,
- `CHAIN` and `COMMON`,
- `PEEK` and `POKE`,
- more than one source file per program,
- inline assembly and object-oriented features.

`PRINT USING` also leaves out the `_` escape, the `%` overflow marker, the `**$` combined prefix and format cycling. Check the reference manual before relying on those.

Build notes for HP-UX

- Build the compiler itself with `gmake`, not the native `make`.
- `bcc` finds its runtime through the `BCCRUNTIME` environment variable, then the directory it was started from, then its install prefix. If `bcc` cannot find `libbasic.a`, set `BCCRUNTIME` to the directory that holds it.

Where to go next

The fastest way to learn BASIC is to build small programs, so pick one of these and write it yourself.

1. **Number guessing game.** The computer picks a number with `RND`, and the player guesses until they get it. Practises `DO ... LOOP`, `INPUT` and `IF`.
2. **Times-table printer.** Ask for a number and print its table with `FOR`. Format it neatly with `PRINT USING`.
3. **Temperature converter.** Write a `FUNCTION` that converts Celsius to Fahrenheit and call it from a loop.
4. **Address book.** Store names and phone numbers in an array of `TYPE` records, and save them with `WRITE #`.
5. **Word counter.** Read a text file and count its lines and words with `EOF`, `INPUT #` and `INSTR`.
6. **Safe divider.** Ask for two numbers and divide them, catching division by zero with `TRY / CATCH`.

When you need exact details, the reference manual (`REFERENCE.md`, installed under `share/doc/hpux-bc`) covers every statement, function and error code. Two debugging switches also help when a program misbehaves:

- `bcc -c file.bas` writes the generated C so you can see what the compiler made of your code.
- `bcc --lex file.bas` and `bcc --parse file.bas` show how the compiler reads your source, which is useful for puzzling syntax errors.